Technical Document #7: Software Engineering - Comprehensive Overview

Technical Document #7: Software Engineering - Comprehensive Overview

API design defines how software components communicate. RESTful APIs and GraphQL are popular API design approaches.

Continuous integration (CI) automatically builds and tests code changes. It catches integration issues early in the development process.

Design patterns provide reusable solutions to common software design problems. Singleton, factory, and observer patterns are widely used.

Refactoring improves code structure without changing behavior. It makes code easier to understand, maintain, and extend.

Technical debt represents the cost of choosing easy solutions over better ones. It accumulates over time and slows development velocity.

Microservices architecture structures applications as independently deployable services. Each service owns its data and business logic.

Sustainability and environmental responsibility are becoming key considerations in technology design. Green computing aims to reduce the environmental impact of technology.

Natural language processing has made significant advances with large language models. These models can understand and generate human-like text with remarkable fluency.

Autonomous vehicles use a combination of sensors, AI, and mapping to navigate without human intervention. Self-driving technology is rapidly advancing.

Federated learning trains models across decentralized data sources without sharing raw data. It enables privacy-preserving machine learning.

Key Takeaways:

- Technical debt represents the cost of choosing easy solutions over better ones.
- Domain-driven design (DDD) models software around business domains.
- Refactoring improves code structure without changing behavior.